

ITP 2 - FINAL PROJECT

Game Statistics Analyzer

Kuhta Igor
Daniyal Anarov
Arman Mukashev
Temirlan Nabi

Problem Description

Games generate lots of player data - but without analysis, it's just numbers. We needed a tool that:

Tracks scores Stores game sessions
with player name, score and date

Shows rankings Leaderboard sorted by
total score across all sessions

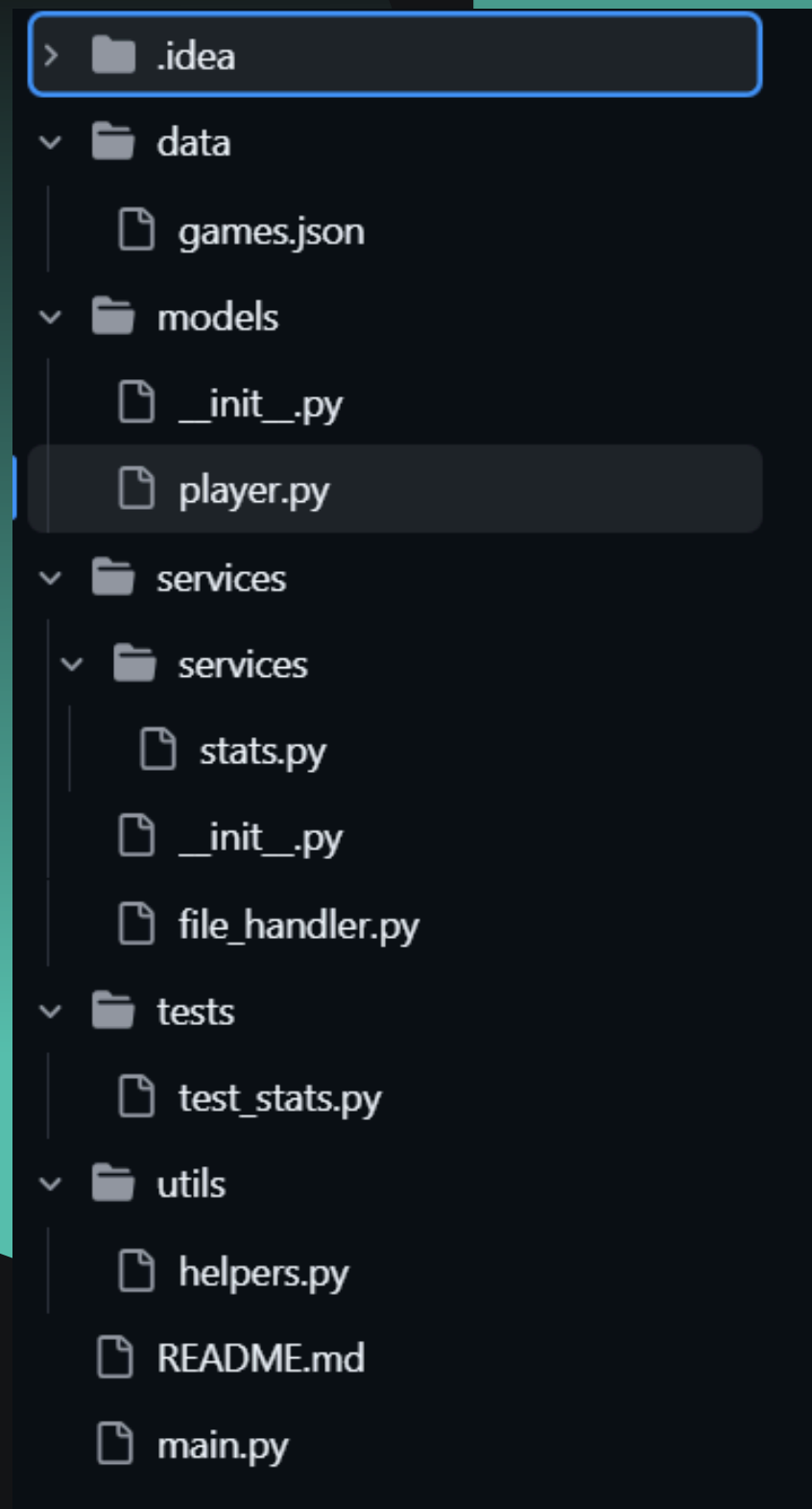
Calculates stats Average score and best
performance per player

Filters data View stats for a specific player

Input format (JSON): [{"player": "Alice", "score": 120, "date": "2026-01-01"}, ...]

System Design

Project Structure



1

models/player.py

GameSessBaseEntity, Player, ion classes

2

services/file_handler.py

load_data(), save_data()
JSON read/write

3

services/stats.py

leaderboard, averages,
best performance, filter

4

main.py utils/ + tests/

Menu, integration,
unit tests, helpers

```
--- Leaderboard ---
1. Dan - 500 points
2. Alice - 300 points
3. Bob - 240 points
4. Charlie - 200 points

=== Game Statistics Analyzer ===
1. Show leaderboard
2. Show average scores
3. Show best performance
4. Show daily best
5. Add new session
6. Filter by player
0. Exit
Choose option:
```

```
--- Average Scores ---
Alice: 150.0
Bob: 120.0
Charlie: 200.0
Dan: 500.0

=== Game Statistics Analyzer ===
1. Show leaderboard
2. Show average scores
3. Show best performance
4. Show daily best
5. Add new session
6. Filter by player
0. Exit
Choose option:
```

```
0. Exit
Choose option: 3

--- Best Performance ---
Dan - 500 points on 2026-05-30

=== Game Statistics Analyzer ===
1. Show leaderboard
2. Show average scores
3. Show best performance
4. Show daily best
5. Add new session
6. Filter by player
0. Exit
Choose option:
```

```
--- Daily Best ---
2026-01-01: Bob - 150 points
2026-01-02: Alice - 180 points
2026-01-03: Charlie - 200 points
2026-05-30: Dan - 500 points

=== Game Statistics Analyzer ===
1. Show leaderboard
2. Show average scores
3. Show best performance
4. Show daily best
5. Add new session
6. Filter by player
0. Exit
Choose option:
```



Leaderboard

Sorted by total score
using lambda + sorted()



Best Performance

Overall and daily
best scores



Averages

Per-player average
with sum() /len()



Add Session

Validates input,
saves to JSON

Team Contribution

Igor
`models/player.py`

- BaseEntity, Player, GameSession
- Inheritance + encapsulation
- Polymorphism (`__str__`)
- `to_dict()` for saving data

OOP

Arman
`services/file_handler.py`

- `load_data()` + `save_data()`
- JSON file processing
- try/except (3 cases)
- Empty file / File not found
- README

File I/O

Temirlan
`services/stats.py`

- `get_leaderboard()` — lambda + sorted
- `get_average_scores()`
- `get_best_performance()`
- `filter_by_player()` — `filter()`

Algorithms

Daniyal
`main.py + utils + tests`

- Menu and module integration
- `helpers.py` (date validation)
- unittest — 8 tests

Integration

Key Decisions & Optimizations

$O(n \log n)$

```
leaderboard = sorted(  
    totals.items(),  
    key=lambda x: x[1],  
    reverse=True )
```

Data Structure Choices

dict

$O(1)$ lookup by player name instead of iterating a list

list

Stores sessions in order for iteration over all data

tuple

(name, score) pairs immutable, from .items()

filter

filter_by_player() instead of nested if in loop

What We Built

- **OOP:** Used classes (BaseEntity, Player, GameSession) with inheritance, encapsulation, and polymorphism (__str__).
- **File I/O:** Managed data via JSON with robust try/except error handling for file issues.
- **Algorithms:** Replaced inefficient bubble sort with sorted() and lambda for better performance ($O(n \log n)$).

- **Python Features:** Utilized efficient built-ins like lambda, filter(), and sum() to replace manual loops.
- **Testing:** Implemented 8 unittest cases covering standard logic and edge cases with automatic setup.
- **Modularity:** Organized into clear folders (models/, services/, utils/) for better project structure.

Thank You

